



UNIVERSIDAD DE CUNDINAMARCA
INGENIERÍA DE SISTEMAS
LINEA DE PROFUNDIZACION III
WILLIAM ALEXANDER MATALLANA PORRAS

CONSULTA DE LAS RELACIONES

PRESENTADO POR:

SERGIO DANIEL CASTELLANOS RODRIGUEZ

PRESENTADO A:

WILLIAM ALEXANDER MATALLANA PORRAS

INGENIERIA EN SISTEMAS

NOVENO SEMESTRE- LINEA DE PROFUNDIZACION III

UNIVERSIDAD DE CUNDINAMARCA

EXTENSION CHIA

ABRIL 2025



Objetivos

Objetivo General

Diseñar e implementar un sistema de gestión académica y logística estudiantil basado en Spring Boot 3.4.4, que integre un modelo relacional avanzado en PostgreSQL mediante Supabase. Este sistema deberá incluir relaciones complejas (1:1, 1:N, N:M), optimizar el código con JPA y Lombok, y garantizar la interoperabilidad con herramientas como pgAdmin4 e IntelliJ IDEA.

Objetivos Específicos

1. Configurar un entorno de desarrollo con:
 - Spring Initializr (Java 21, Maven, packaging JAR).
 - Conexión a Supabase como proveedor de PostgreSQL.
 - Integración de dependencias: Spring Web, DevTools, Data JPA, PostgreSQL Driver.
2. Modelar relaciones de base de datos en Supabase:
 - **1:1**: Estudiante-MedioTransporte.
 - **1:N**: Profesor-Estudiante.
 - **N:M**: Estudiante-Materia y Estudiante-EquipoDeportivo.
3. Implementar el mapeo objeto-relacional (JPA) con anotaciones avanzadas.
4. Optimizar entidades con Lombok para eliminar código redundante.
5. Validar el modelo mediante consultas SQL en pgAdmin4 y Supabase Studio.

Actividades Detalladas

1. Configuración de Supabase

- 1.1. Crear proyecto en supabase.com.
- 1.2. Generar contraseña segura para el rol postgres.
- 1.3. Configurar conexión en application.properties usando variables de entorno.

2. Diseño de Relaciones

2.1. Uno a Uno (1:1):

- Tabla medio_transporte con FK a estudiante.codigo.

2.2. Uno a Muchos (1:N):



- Tabla profesor vinculada a estudiante vía profesor_id.
- 2.3. **Muchos a Muchos (N:M):**
- Tablas pivote estudiante_materia y estudiante_equipo_deportivo.

3. Implementación Lombok

Desarrollo de la actividad

Configuración de Supabase

-Crear proyecto en Supabase:

- Accede a supabase.com y crea un nuevo proyecto.
- En la sección "Database", genera una contraseña segura para el rol postgres.

-Obtener credenciales de conexión:

- Copia la URL de conexión, el usuario y la contraseña proporcionados por Supabase.

```
1 BD_URL=jdbc:postgresql://aws-0-us-east-1.pooler.supabase.com:5432/postgres
2 BD_USERNAME=postgres.ybojcvyldvwmgyngkth
3 BD_PASSWORD=XTuj6aXhZ8G3tz3H
```

-Configurar variables de entorno:

- Almacena las credenciales en un archivo .env y carga las variables en BdInitApplication.java.

1. Relación Uno a Uno (1:1)

Objetivo: Vincular un estudiante con un único medio de transporte.

Clase Involucrada: MedioTransporte.java.

Implementación:



```
@Entity new *
@Table(name = "medio_transporte")
@Data
@NoArgsConstructor
public class MedioTransporte {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String tipo; // Ej: "Bicicleta", "Automóvil"

    @OneToOne
    @JoinColumn(name = "estudiante_id", unique = true)
    @JsonIgnoreProperties("medioTransporte")
    private Estudiante estudiante;
}
```

Explicación:

- **@OneToOne**: Define la relación 1:1.
- **@JoinColumn**: Indica que estudiante_id es la columna de unión en la tabla medio_transporte.
- **@JsonIgnoreProperties**: Evita ciclos infinitos al serializar a JSON.

Tabla en Supabase:

```
CREATE TABLE medio_transporte (
  id SERIAL PRIMARY KEY,
  tipo VARCHAR(50) NOT NULL,
  estudiante_id INT UNIQUE REFERENCES estudiante(codigo)
);
```

2. Relación Uno a Muchos (1:N)

Objetivo: Un profesor puede tener múltiples estudiantes.

Clase Involucrada: Profesor.java.



Implementación:

```
@Entity new *
public class Profesor {
    @Id
    private long codigo;
    private String nombre; 4 usages
    private String telefono; 4 usages

    @OneToMany(mappedBy = "profesor", cascade = CascadeType.ALL) no usages
    private List<Estudiante> estudiantes;
```

Explicación:

- **@OneToMany**: Indica que un profesor tiene muchos estudiantes.
- **mappedBy**: Define que la relación está controlada por el campo profesor en Estudiante.java.

Tabla en Supabase:

```
CREATE TABLE profesor (
    codigo BIGINT PRIMARY KEY,
    nombre VARCHAR(100),
    telefono VARCHAR(20)
);
```

3. Relación Muchos a Muchos (N:M)

Objetivo: Un estudiante puede pertenecer a varios equipos deportivos.

Clase Involucrada: EquipoDeportivo.java.

Implementación:



```
@NoArgsConstructor
public class EquipoDeportivo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String nombre;

    @ManyToMany
    @JoinTable(
        name = "estudiante_equipo",
        joinColumns = @JoinColumn(name = "equipo_id"),
        inverseJoinColumns = @JoinColumn(name = "estudiante_id")
    )
    @JsonIgnoreProperties("equipos") // Evita recursión en JSON
    private List<Estudiante> miembros;
```

Explicación:

- **@ManyToMany**: Define la relación N:M.
- **@JoinTable**: Crea la tabla intermedia estudiante_equipo con las claves foráneas.

Tabla en Supabase:

```
CREATE TABLE estudiante_equipo (
    estudiante_id BIGINT REFERENCES estudiante(codigo)
    equipo_id BIGINT REFERENCES equipo_deportivo(id),
    PRIMARY KEY (estudiante_id, equipo_id)
);
```



Anotaciones Clave

Anotaciones de JPA

Anotación	Descripción	Ejemplo
@Entity	Marca la clase como una entidad de base de datos.	@Entity public class Estudiante { ... }
@Id	Marca la clase como una entidad de base de datos.	@Id private Long id;
@GeneratedValue	Especifica la estrategia de generación de valores para la clave primaria.	@GeneratedValue(strategy = GenerationType.IDENTITY)
@OneToOne	Establece una relación 1:1 entre entidades.	@OneToMany(mappedBy = "profesor") private List<Estudiante> estudiantes;
@OneToMany	Define una relación 1:N.	@OneToOne @JoinColumn(name = "estudiante_id")
@ManyToMany	Crea una relación N:M.	@ManyToMany @JoinTable(name = "estudiante_materia")
@Column	Personaliza detalles de una columna (nombre, nulidad, longitud).	@Column(name = "nombre", nullable = false, length = 100)
@Table	Define el nombre de la tabla en la base de datos.	@Table(name = "estudiantes")
@JoinColumn	Indica la columna de unión en la base de datos.	@JoinColumn(name = "profesor_id")
@JoinTable	Define la tabla intermedia para relaciones N:M.	JoinTable(name = "estudiante_equipo")



Anotaciones de Lombok

Anotación	Descripción	Ejemplo
@Getter/@Setter	Genera métodos para acceder y modificar campos.	@Getter @Setter private String nombre;
@ToString	Genera el método toString() automáticamente.	@ToString public class Estudiante { ... }
@EqualsAndHashCode	Implementa equals() y hashCode() basados en los campos.	@EqualsAndHashCode public class Profesor { ... }
@RequiredArgsConstructor	Crea un constructor con campos obligatorios (@NonNull o final).	@RequiredArgsConstructor public class Materia { ... }
@Data	Combina @Getter, @Setter, @ToString, @EqualsAndHashCode y @RequiredArgsConstructor.	@Data public class Estudiante { ... }
@NoArgsConstructor	Genera un constructor sin argumentos (requerido por JPA).	@NoArgsConstructor public class Estudiante { ... }
@AllArgsConstructor	Genera un constructor con todos los campos.	@AllArgsConstructor public class Profesor { ... }
@NonNull	Marca un campo como no nulo (error si se asigna null).	@NonNull private String nombre;

Ejemplo Comparativo: JPA vs. Lombok

Sin Lombok (JPA tradicional):

```
@Entity new *
@Entity
public class Estudiante {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre; 2 usages

    // Constructor vacío manual
    public Estudiante() {} new *

    // Getters y Setters manuales
    public Long getId() { return id; } no usages new *
    public void setId(Long id) { this.id = id; } no usages new *
    public String getNombre() { return nombre; } no usages new *
    public void setNombre(String nombre) { this.nombre = nombre; } no usages new *
```




Con Lombok:

```
10 @Entity new *
11 @Table(name = "estudiante")
12 @Data
13 @NoArgsConstructor
14 public class Estudiante {
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private long codigo;
18     private String nombre;
19     private String telefono;
20 }
```

Modelo en Supabase

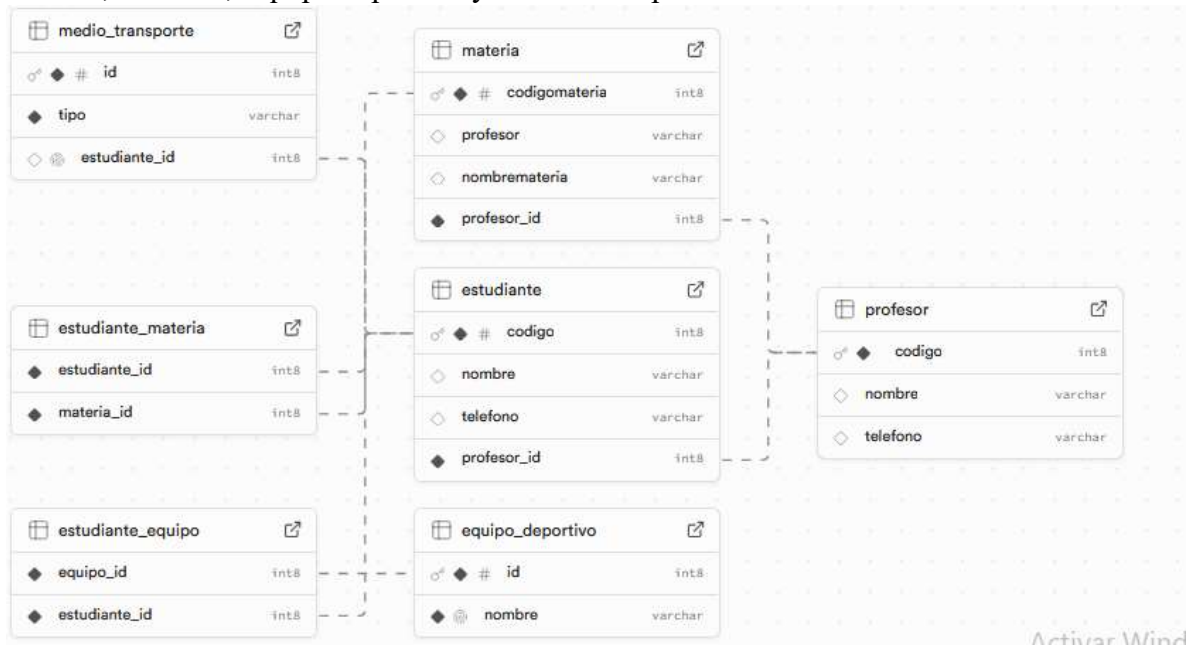
El modelo generado incluye las siguientes tablas:

1. **estudiante:** Almacena información de estudiantes.
2. **profesor:** Contiene datos de profesores y su relación con estudiantes.
3. **materia:** Registra materias y su vinculación con estudiantes.
4. **equipo_deportivo:** Gestiona equipos y sus miembros.
5. **medio_transporte:** Relaciona estudiantes con su medio de transporte.

Como se ha mostrado anteriormente, la base del ejercicio son cuatro entidades, Estudiante,



materia, Profesor, EquipoDeportivo y MedioTransporte.



siendo la estructura de la sentidades las siguientes



Estudiante

```
1 //Codigo elaborado por Sergio Daniel Castellanos Rodriguez
2 package com.example.bd_init.model;
3
4 import jakarta.persistence.*;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
8 import java.util.List;
9
10 @Entity
11 @Table(name = "estudiante")
12 @Data
13 @NoArgsConstructor
14 public class Estudiante {
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private long codigo;
18     private String nombre;
19     private String telefono;
20
21     @ManyToOne
22     @JoinColumn(name = "profesor_id", nullable = false)
23     @JsonIgnoreProperties("estudiantes") // Evita recursión JSON
24     private Profesor profesor;
25
26     @ManyToMany(cascade = CascadeType.ALL)
27     @JoinTable(
28         name = "estudiante_materia",
29         joinColumns = @JoinColumn(name = "estudiante_id"),
30         inverseJoinColumns = @JoinColumn(name = "materia_id")
31     )
32     @JsonIgnoreProperties("estudiantes")
33     private List<Materia> materias;
34
35     // Relación 1:1 con MedioTransporte
36     @OneToOne(mappedBy = "estudiante", cascade = CascadeType.ALL)
37     @JsonIgnoreProperties("estudiante")
38     private MedioTransporte medioTransporte;
39 }
```



Materia

```
1 //Codigo elaborado por Sergio Daniel Castellanos Rodriguez
2 package com.example.bd_init.model;
3 import jakarta.persistence.*;
4
5 import java.util.List;
6
7 @Entity
8 public class Materia {
9     @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private long codigomateria;
12     private String nombremateria;
13     private String profesor;
14
15     @ManyToOne
16     @JoinColumn(name = "profesor_id", nullable = false)
17     private Profesor profesor;
18
19     @ManyToMany(mappedBy = "materias")
20     private List<Estudiante> estudiantes;
21
22     public Materia() {}
23
24     public Materia(long codigomateria, String nombremateria, String profesor) {}
25
26     this.codigomateria = codigomateria;
27     this.nombremateria = nombremateria;
28     this.profesor = profesor;
29
30     public long getCodigomateria() {}
31     return codigomateria;
32
33     public void setCodigomateria(long codigomateria) {}
34     this.codigomateria = codigomateria;
35
36     public String getNombremateria() {}
37     return nombremateria;
38
39     public void setNombremateria(String nombremateria) {}
40     this.nombremateria = nombremateria;
41
42     public String getProfesor() {}
43     return profesor;
44
45     public void setProfesor(String profesor) {}
46     this.profesor = profesor;
47
48 }
```

Profesor



UNIVERSIDAD DE CUNDINAMARCA
INGENIERÍA DE SISTEMAS
LINEA DE PROFUNDIZACION III
WILLIAM ALEXANDER MATAALLANA PORRAS

```
24     }
25
26     public Professor(long codigo, String nombre, String telefono) { no usages new *
27         this.codigo = codigo;
28         this.nombre = nombre;
29         this.telefono = telefono;
30     }
31
32     public long getCodigo() { no usages new *
33         return codigo;
34     }
35
36     public void setCodigo(long codigo) { no usages new *
37         this.codigo = codigo;
38     }
39
40     public String getNombre() { no usages new *
41         return nombre;
42     }
43
44     public void setNombre(String nombre) { no usages new *
45         this.nombre = nombre;
46     }
47
48
49 //codigo elaborado por Sergio Daniel Castellanos Rodriguez
50 package com.example.bd_init.model;
51
52 import jakarta.persistence.CascadeType;
53 import jakarta.persistence.Entity;
54 import jakarta.persistence.Id;
55 import jakarta.persistence.OneToMany;
56
57 import java.util.List;
58
59 @Entity no usages
60 public class Profesor {
61     @Id
62     private long codigo;
63     private String nombre; 4 usages
64     private String telefono; 4 usages
65
66     @OneToMany(mappedBy = "profesor", cascade = CascadeType.ALL) no usages
67     private List<Estudiante> estudiantes;
68
69     @OneToMany(mappedBy = "profesor", cascade = CascadeType.ALL) no usages
70     private List<Materia> materias;
71     public Profesor() { no usages
72     }
73
74     public String getTelefono() { no usages new *
75         return telefono;
76     }
77
78     public void setTelefono(String telefono) { no usages new *
79         this.telefono = telefono;
80     }
81
82     @Override no usages
83     public String toString() {
84         return "Profesor{" +
85             "codigo=" + codigo +
86             ", nombre=" + nombre + '\n' +
87             ", telefono=" + telefono + '\n' +
88             '}';
89     }
90 }
```



EquipoDeportivo

```
1 //Codigo elaborado por Sergio Daniel Castellanos Rodriguez
2 package com.example.bd_init.model;
3
4 import jakarta.persistence.*;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7 import java.util.List;
8 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
9
10 /**
11  * Representa un equipo deportivo al que pueden pertenecer múltiples estudiantes.
12  * Relación N:M con Estudiante mediante tabla intermedia.
13  * @author (Sergio Daniel Castellanos Rodriguez)
14  * @see <a href="https://docs.jboss.org/hibernate/orm/5.4/userguide/html_single/Hibernate_User_Guide.html#assoc">
15  */
16 @Entity(name = "equipo_deportivo")
17 @Table(name = "equipo_deportivo")
18 @Data
19 @NoArgsConstructor
20 public class EquipoDeportivo {
21
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     private Long id;
25
26     @Column(nullable = false, unique = true)
27     private String nombre;
28
29     @ManyToMany
30     @JoinTable(
31         name = "estudiante_equipo",
32         joinColumns = @JoinColumn(name = "equipo_id"),
33         inverseJoinColumns = @JoinColumn(name = "estudiante_id")
34     )
35     @JsonIgnoreProperties("equipos") // Evite recursión en JSON
36     private List<Estudiante> miembros;
37 }
```

MedioTransporte



```
1 //Codigo elaborado por Sergio Daniel Castellanos Rodriguez
2 package com.example.bd_init.model;
3
4 import jakarta.persistence.*;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
8
9 /**
10  * Modelo #1 medio de transporte personal de un estudiante.
11  * @author [Sergio Daniel Castellanos Rodriguez]
12  * @see <a href="https://www.baeldung.com/jpa-one-to-one">Baeldung 1:1 Guide</a>
13  */
14 @Entity
15 @Table(name = "medio_transporte")
16 @Data
17 @NoArgsConstructor
18 public class MedioTransporte {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Long id;
23
24     @Column(nullable = false)
25     private String tipo; // Ej: "Bicicleta", "Automóvil"
26
27     @OneToOne
28     @JoinColumn(name = "estudiante_id", unique = true)
29     @JsonIgnoreProperties("medioTransporte")
30     private Estudiante estudiante;
31 }
```

Referencias

- Oracle. (2023). *Project Lombok*. <https://projectlombok.org/>
 - Baeldung. (2023). *JPA One-to-One Relationship*. <https://www.baeldung.com/jpa-one-to-one>
 - Supabase. (2023). *PostgreSQL Connection Docs*. <https://supabase.com/docs/guides/database>
 - Hibernate. (2023). *Mapping Entity Associations*. [https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/Hibernate User Guide.html](https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/Hibernate%20User%20Guide.html)
-



UNIVERSIDAD DE CUNDINAMARCA
INGENIERÍA DE SISTEMAS
LINEA DE PROFUNDIZACION III
WILLIAM ALEXANDER MATALLANA PORRAS